So far, you have created a Deployment for a Spring Boot application and run it in your local Kubernetes cluster. But it's not possible to use it yet, since it's isolated inside the cluster. In the next section you'll learn how to expose your application to the outside world and how to use the service-discovery and load-balancing functionality provided by Kubernetes.

7.3 Service discovery and load balancing

We've talked about Pods and Deployments, so let's dig into Services a bit. You've got the Catalog Service application running as a Pod in your local Kubernetes cluster, but there are still unanswered questions. How can it interact with the PostgreSQL Pod running in the cluster? How does it know where to find it? How can you expose a Spring Boot application to be used by other Pods in the cluster? How can you expose it outside the cluster?

This section will answer those questions by introducing two important aspects of cloud native systems: service discovery and load balancing. I'll present the two main patterns available to implement them when working with Spring applications: client-side and server-side. Then you'll apply the latter approach, which is conveniently offered natively by Kubernetes through Service objects, meaning you don't have to change anything in your code to support it (unlike the client-side option). Finally, you'll learn how the communication between the Catalog Service Pod and the PostgreSQL Pod happens, and you'll expose the Catalog Service application as a network service.

7.3.1 Understanding service discovery and load balancing